CME Futures Microstructure Analysis Pipeline

Architecture & Design Document — v3 (Final)

---

1. Confirmed Data Schema

CLOB Parquet — Columns

```
seq_num          int64      — sequence number (shared key with transactions)
timestamp_ms     int64      — epoch milliseconds (or timestamp[ms])
product_id       string     — e.g. "ES", "NQ", "CL"
bid_px1          float64    — best bid price
bid_sz1          float64    — best bid size (contracts)
ask_px1          float64    — best ask price
ask_sz1          float64    — best ask size
bid_px2..bid_px10  float64  — levels 2–10 bid prices
bid_sz2..bid_sz10  float64  — levels 2–10 bid sizes
ask_px2..ask_px10  float64  — levels 2–10 ask prices
ask_sz2..ask_sz10  float64  — levels 2–10 ask sizes
```

Total: 1 (seq) + 1 (ts) + 1 (product) + 40 (10 levels × 4 columns) = **43 columns**

Transaction Parquet — Columns

```
seq_num          int64      — sequence number (shared key with CLOB)
timestamp_ms     int64      — epoch milliseconds
product_id       string     — e.g. "ES", "NQ", "CL"
price            float64    — trade price
qty              float64    — trade quantity (contracts)
```

Total: **5 columns** (aggressor side will be inferred)

---

2. Optimal Partitioning Strategy

Recommendation: `date / product` dual partition for CLOB

```
clob/
  date=2024-01-15/
    product=ES/
```

```
      part-00000.parquet      ← one or few files per product per day
      part-00001.parquet
    product=NQ/
      part-00000.parquet
    product=CL/
      ...
  date=2024-01-16/
    ...

transactions/
  date=2024-01-15/
    product=ES/
      part-00000.parquet
    product=NQ/
      ...
```

Why this layout:

Consideration	Decision	Reasoning
Partition by date first	✅	Every pipeline run processes exactly one day; date partition gives instant directory-level pruning
Partition by product second	✅	Pipeline processes one product at a time; product partition means PyArrow reads ONLY that product's files — no scanning/filtering 99 irrelevant products
Single file per day (no product partition)	❌	Would force PyArrow to scan all 100 products and filter, wasting I/O bandwidth; for a 100 GB daily CLOB file, this wastes ~99% of reads
Product first, date second	❌	Viable but less natural; date-first matches the "daily batch" access pattern better
Sort within each file	✅ by seq_num	Enables row-group min/max pruning for sequence-based lookups and guarantees the interleave merge works correctly

Parquet file settings per partition:

```
Row group size:    1,000,000 rows
Compression:       ZSTD level 3
Write statistics:  ON (enables seq_num / timestamp_ms pruning)
```

```
Dictionary:      OFF for float64; ON for product_id (if embedded)
Sort order:      By seq_num ascending within each file
```

Sizing per product per day:

A liquid product like ES may have 20–50M CLOB updates/day.

- 50M rows × 43 columns × 8 bytes = ~17 GB uncompressed
- ZSTD compressed: ~3–5 GB per product per day
- 100 products: ~100–300 GB per day (total CLOB)

With `date/product` partitioning, each pipeline invocation reads only one product's 3–5 GB partition — fits comfortably in the I/O budget.

---

3. Revised Pipeline Architecture

Processing model: One product at a time, full day in memory

```
128 GB RAM budget:
  Single product CLOB arrays (50M updates × 43 cols × 8B)  ≈ 17 GB peak
  Transaction arrays (2M trades × 5 cols × 8B)             ≈  0.08 GB
  Millisecond timeline (82.8M slots × 15 metrics × 8B)     ≈  10 GB
  Event windows (500K events × 14 offsets × 15 metrics)    ≈  0.8 GB
  Working buffers                                          ≈  4 GB

  ──────────────────────────────────────────────────────────────────
  Peak per product                                         ≈  32 GB
  Remaining headroom                                       ≈  96 GB ✅
```

Full pipeline flow:

```
┌──────────────────────────────────────────────────────────────────────┐
│  ORCHESTRATOR:  for each product in products_list:                   │
│                 run_single_product(date, product)                    │
│                 gc.collect()  # release arrays before next           │
└──────────────────────────────────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────────┐
│  PHASE A: LOAD & INTERLEAVE                                          │
│                                                                      │
```

```
│ A1. Read CLOB partition:                                    │
│   pq.read_table("clob/date=.../product=ES/")                 │
│   → Already sorted by seq_num within files                   │
│   → Extract: seq_num, timestamp_ms, bid/ask px/sz (10 levels)  │
│   → Convert to NumPy arrays (zero-copy for numeric columns)    │
│                                                              │
│ A2. Read Transaction partition:                              │
│   pq.read_table("transactions/date=.../product=ES/")          │
│   → Extract: seq_num, timestamp_ms, price, qty               │
│   → Convert to NumPy arrays                                  │
│                                                              │
│ A3. Interleave by seq_num (Numba merge):                     │
│   Combined stream ordered by seq_num lets us walk through     │
│   book updates and trades in exact exchange sequence.         │
│   For each transaction, the "prior book state" is the last    │
│   CLOB record with seq_num < trade.seq_num.                   │
│                                                              │
│ A4. Infer aggressor side + detect sweeps (Numba sequential scan):  │
│   Walk interleaved stream:                                   │
│     - CLOB record → update running best_bid, best_ask         │
│     - Transaction record →                                   │
│         if price >= best_ask: aggressor = BUY                 │
│         if price <= best_bid: aggressor = SELL                │
│         if price > best_ask:  swept = True  (bought through L1)  │
│         if price < best_bid:  swept = True  (sold through L1)    │
│   Output: aggressor_side[], swept_flag[] per transaction      │
└──────────────────────────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│ PHASE B: BUILD MILLISECOND TIMELINE                          │
│                                                              │
│ The trading day has 82,800,000 millisecond slots             │
│ (23 hours × 3,600,000 ms/hour, from 17:00 CST to 16:00 CST)   │
│                                                              │
│ B1. Map each CLOB update to its ms slot:                     │
│   ms_index = (timestamp_ms - day_start_ms)                   │
│   If multiple CLOB updates in same ms → keep LAST by seq_num  │
│                                                              │
│ B2. Forward-fill order book (Numba sequential scan):         │
│   For each ms slot 0..82,799,999:                            │
│     if CLOB update exists at this ms → use it                 │
│     else → carry forward previous ms values                   │
│   Result: 10 contiguous arrays of 82.8M (one per book metric)  │
```

```
│      book_bid_px1[82.8M], book_bid_sz1[82.8M], ...              │
│      book_ask_px1[82.8M], book_ask_sz1[82.8M], ...              │
│      (levels 1 and 2 used for metrics; all 10 available)        │
│                                                                  │
│  B3. Bucket transactions into ms slots (Numba scan):             │
│    For each ms slot, aggregate all transactions that fell in it: │
│      vwap[ms]            = Σ(px×qty) / Σ(qty)                    │
│      num_transactions[ms] = count                                │
│      aggressor_buy_vol[ms]  = Σ qty where side=BUY               │
│      aggressor_sell_vol[ms] = Σ qty where side=SELL              │
│      swept_book[ms]       = 1 if ANY trade in this ms was tagged │
│    Slots with no transactions: NaN for vwap, 0 for counts        │
└──────────────────────────────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────┐
│  PHASE C: COMPUTE STEP 1 METRICS (Numba parallel)                │
│                                                                  │
│  For all 82.8M ms slots simultaneously:                          │
│                                                                  │
│  ORDER BOOK METRICS (from forward-filled arrays):                │
│    mid_price[ms]        = (bid_px1 + ask_px1) / 2                │
│    bid_ask_spread[ms]    = ask_px1 - bid_px1                     │
│    level1_bid_depth[ms]  = bid_sz1                               │
│    level1_ask_depth[ms]  = ask_sz1                               │
│    level2_bid_depth[ms]  = bid_sz2                               │
│    level2_ask_depth[ms]  = ask_sz2                               │
│                                                                  │
│  TRANSACTION METRICS (from bucketed arrays, computed in B3):     │
│    vwap[ms]                                                      │
│    num_transactions[ms]                                          │
│    aggressor_buy_vol[ms]                                         │
│    aggressor_sell_vol[ms]                                        │
│    swept_book[ms]                                                │
│                                                                  │
│  Total: 11 metric arrays, each 82.8M elements                    │
│                                                                  │
│  *** OPTIONAL: Write Step 1 to Parquet ***                       │
│  step1/product=ES/date=2024-01-15.parquet                        │
│  (82.8M rows × 11 metrics ≈ 1.5–3 GB compressed)                 │
│  Default: SKIP (not needed if Steps 2–3 computed in same pass)   │
└──────────────────────────────────────────────────────────────────┘
                    │
                    ▼
```

```
PHASE D: EXTRACT EVENT WINDOWS — Step 2 (Numba parallel)

D1. Find event milliseconds:
    event_indices = np.where(num_transactions > 0)[0]
    Example: 500,000 events for a liquid product

D2. Gather 14-offset windows (Numba prange over events):
    OFFSETS = [-5, -4, -3, -2, -1, 0, +1, +2, +3, +4, +5,
               +6, +7, +8]

    for event_idx in prange(num_events):
        t = event_indices[event_idx]
        for k, offset in enumerate(OFFSETS):
            src = t + offset
            if 0 <= src < 82_800_000:
                for m in range(num_metrics):
                    windows[event_idx, k, m] = metrics[m][src]

D3. Tag each event with its hour:
    hour_of_event = (event_indices // 3_600_000) + start_hour
    (start_hour = 17 for 17:00 CST, wraps at 24 → 0)

D4. Write Step 2 to Parquet:
    step2/product=ES/date=2024-01-15.parquet
    Columns: event_ms, hour, offset, mid_price, spread, ...
    Rows: 500K events × 14 offsets = 7M rows
    Size: ~150–300 MB compressed
```

```
▼
```

```
PHASE E: HOURLY PERCENTILES — Step 3 (NumPy)

For each hour h in [17, 18, ..., 23, 0, 1, ..., 15, 16]:
  For each offset k in [-5 .. +8]:
    For each metric m:
      values = windows[hour==h, offset==k, metric==m]
      percentiles = np.percentile(values, [5, 25, 50, 75, 95])

Output schema:
  product   string
  date      date32
  hour      int8      (0–23, CST)
```

```
│   offset     int8       (-5 to +8)                          │
│   metric     string     (metric name)                       │
│   p5         float64                                        │
│   p25        float64                                        │
│   p50        float64                                        │
│   p75        float64                                        │
│   p95        float64                                        │
│   n_events   int32      (count of events in this hour)      │
│                                                             │
│  Rows: 23 hours × 14 offsets × 11 metrics = 3,542 rows      │
│  Write: step3/product=ES/date=2024-01-15.parquet (tiny, <1 MB)  │
└─────────────────────────────────────────────────────────────┘
                    │
                    ▼
┌─────────────────────────────────────────────────────────────┐
│  PHASE F: MULTI-DAY AGGREGATION — Step 4                    │
│                                                             │
│  Run AFTER all daily pipelines complete for the date range. │
│                                                             │
│  F1. For each product:                                      │
│     Read ALL step2 Parquet files for the date range         │
│     (e.g., 63 days × 300 MB = ~19 GB for one liquid product)  │
│                                                             │
│  F2. Pool events across all days:                           │
│     For each hour:                                          │
│       For each offset:                                      │
│         For each metric:                                    │
│           values = ALL events across ALL days in this hour+offset  │
│           percentiles = np.percentile(values, [5,25,50,75,95])  │
│                                                             │
│  F3. Write: step4/product=ES/Q1_2024.parquet                │
│     Same schema as Step 3, but date column = NULL or date_range  │
│                                                             │
│  F4. Generate fan charts:                                   │
│     For each product × each hour × each metric:             │
│       x-axis:  14 offsets [-5, -4, ..., 0, ..., +8]         │
│       bands:   P5–P95 (lightest), P25–P75 (medium), P50 (line)  │
│       Save:    charts/product=ES/hour=08/metric_name.png    │
│                                                             │
│  Memory strategy for pooling:                               │
│   Option A: Brute force — read all days into memory (~19 GB)  │
│             Feasible with 128 GB RAM.                       │
│   Option B: Streaming — accumulate per (hour, offset, metric)  │
│             using t-digest for approximate percentiles.     │
```

> Recommendation: Option A for simplicity. Fall back to B if a product has extreme event counts.

---

4. Sweep Detection — Detailed Logic

The sweep detection happens in Phase A4, walking the interleaved seq_num stream. This is the most subtle part of the pipeline.

```
Algorithm: infer_aggressor_and_sweep()

State:
  running_best_bid = NaN
  running_best_ask = NaN

For each record in interleaved stream (ordered by seq_num):

  if record is CLOB update:
      running_best_bid = record.bid_px1
      running_best_ask = record.ask_px1

  if record is TRANSACTION:
      # Aggressor inference
      if record.price >= running_best_ask:
          aggressor = BUY      # buyer lifted the offer
      elif record.price <= running_best_bid:
          aggressor = SELL     # seller hit the bid
      else:
          aggressor = UNKNOWN  # mid-price trade or stale book

      # Sweep detection
      if record.price > running_best_ask:
          swept = True         # paid MORE than best ask → consumed L1+
      elif record.price < running_best_bid:
          swept = True         # sold BELOW best bid → consumed L1+
      else:
          swept = False

      # Store results
      trade_aggressor[trade_index] = aggressor
      trade_swept[trade_index] = swept
      trade_index += 1
```

Key distinction:

- `price >= best_ask` → aggressor BUY (normal aggressive buy, may or may not sweep)
- `price > best_ask` → swept book (strictly greater means it went past level 1)
- `price == best_ask` → aggressive buy but did NOT sweep (filled at top of book)

Same logic mirrored for sells.

---

5. Complete Metric Catalog

#	Metric Name	Source	Type	Per-ms Logic
1	mid_price	Book	float64	(bid_px1 + ask_px1) / 2
2	bid_ask_spread	Book	float64	ask_px1 - bid_px1
3	level1_bid_depth	Book	float64	bid_sz1
4	level1_ask_depth	Book	float64	ask_sz1
5	level2_bid_depth	Book	float64	bid_sz2
6	level2_ask_depth	Book	float64	ask_sz2
7	vwap	Trades	float64	Σ(px×qty)/Σ(qty), NaN if no trades
8	num_transactions	Trades	int32	count of trades in this ms
9	aggressor_buy_vol	Trades	float64	Σ qty where aggressor=BUY
10	aggressor_sell_vol	Trades	float64	Σ qty where aggressor=SELL
11	swept_book	Trades	int8	1 if any trade in this ms was a sweep, else 0

---

6. Parquet Output Schemas

Step 1 (Optional — 82.8M rows per product per day)

```
step1/product={product}/date={date}.parquet

ms_index          int32     — millisecond offset from day start (0..82,799,999)
timestamp_ms      int64     — absolute epoch ms
mid_price         float64
bid_ask_spread    float64
level1_bid_depth  float64
level1_ask_depth  float64
level2_bid_depth  float64
level2_ask_depth  float64
vwap              float64   — NaN if no trades
num_transactions  int32
```

```
aggressor_buy_vol float64
aggressor_sell_vol float64
swept_book        int8
```

Step 2 (Event Windows — ~7M rows per product per day)

```
step2/product={product}/date={date}.parquet

event_ms          int32     — ms index of the triggering transaction
hour              int8      — hour of day (CST: 17,18,...,23,0,1,...,16)
offset            int8      — window offset (-5 to +8)
mid_price         float64
bid_ask_spread    float64
level1_bid_depth  float64
level1_ask_depth  float64
level2_bid_depth  float64
level2_ask_depth  float64
vwap              float64
num_transactions  int32
aggressor_buy_vol float64
aggressor_sell_vol float64
swept_book        int8
```

Step 3 (Hourly Percentiles — ~3,542 rows per product per day)

```
step3/product={product}/date={date}.parquet

hour              int8
offset            int8
metric            string    — metric name
p5                float64
p25               float64
p50               float64
p75               float64
p95               float64
n_events          int32     — number of events in this hour
```

Step 4 (Multi-day Aggregated — ~3,542 rows per product per period)

```
step4/product={product}/{period}.parquet

hour              int8
```

```
offset        int8
metric        string
p5            float64
p25           float64
p50           float64
p75           float64
p95           float64
n_events      int32     — total events pooled across all days
date_start    date32    — first date in range
date_end      date32    — last date in range
n_days        int16     — number of trading days pooled
```

---

7. Performance Estimates (Revised)

Per product, per day:

Phase	Operation	Time Est.	Bottleneck
A1	Read CLOB Parquet (3–5 GB)	3–6s	Disk I/O
A2	Read Transactions (<100 MB)	<1s	Disk I/O
A3	Interleave by seq_num	2–4s	Numba sequential
A4	Infer aggressor + sweep	1–2s	Numba sequential
B1-B2	Build ms timeline + forward fill	3–5s	Numba sequential (82.8M slots)
B3	Bucket transactions	1–2s	Numba sequential
C	Compute book metrics	1–2s	Numba parallel
D	Gather event windows	1–2s	Numba parallel
E	Hourly percentiles	<1s	NumPy
Write	Step 2 + Step 3 Parquet	2–4s	Disk I/O
Total		**15–30s**	

100 products, 1 day: 25–50 minutes

1 quarter (63 days): 26–52 hours

Parallelization opportunity:

Different days are fully independent. With 2–4 machines (or tmux sessions processing different date ranges), a quarter can finish in 6–13 hours.

---

8. Directory Structure — Complete

```
project/
├── config/
│   └── pipeline_config.yaml      — products list, paths, parameters
│
├── src/
│   ├── __init__.py
│   ├── config.py                 — PipelineConfig dataclass
│   ├── io_utils.py               — Parquet read/write helpers
│   ├── numba_kernels.py          — All @njit functions
│   ├── phase_a_load.py           — Load, interleave, infer aggressor
│   ├── phase_b_timeline.py       — Build ms timeline, forward fill
│   ├── phase_c_metrics.py        — Compute Step 1 metrics
│   ├── phase_d_windows.py        — Extract event windows (Step 2)
│   ├── phase_e_percentiles.py    — Hourly percentiles (Step 3)
│   ├── phase_f_aggregate.py      — Multi-day aggregation (Step 4)
│   ├── fan_chart.py              — Matplotlib fan chart generator
│   ├── daily_pipeline.py         — Orchestrate Phases A–E for 1 day
│   └── quarterly_pipeline.py     — Orchestrate Phase F across days
│
├── data/
│   ├── raw/
│   │   ├── clob/
│   │   │   └── date=YYYY-MM-DD/product=XX/*.parquet
│   │   └── transactions/
│   │       └── date=YYYY-MM-DD/product=XX/*.parquet
│   │
│   ├── step1/  (optional)
│   │   └── product=XX/date=YYYY-MM-DD.parquet
│   ├── step2/
│   │   └── product=XX/date=YYYY-MM-DD.parquet
│   ├── step3/
│   │   └── product=XX/date=YYYY-MM-DD.parquet
│   └── step4/
│       └── product=XX/{period}.parquet
│
```

```
├── charts/
│   └── product=XX/hour=HH/
│       ├── mid_price.png
│       ├── bid_ask_spread.png
│       └── ...
│
└── tests/
    ├── test_numba_kernels.py
    ├── test_sweep_detection.py
    └── generate_synthetic_data.py
```

---

9. Numba Kernel Inventory

Kernel	Parallel?	Why
merge_interleave_by_seq()	No	Must preserve global seq order
infer_aggressor_and_sweep()	No	Stateful: tracks running BBO
map_to_ms_slots()	No	Ordered scan for last-per-ms
forward_fill_book()	No	Each slot depends on previous
bucket_transactions()	No	Accumulate into ms bins
compute_book_metrics()	Yes	Independent per ms slot
gather_event_windows()	Yes	Independent per event

The sequential kernels (interleave, forward fill, bucket) are O(n) single-pass scans — very cache-friendly and fast despite being single-threaded.

The parallel kernels (metrics, window gather) use `nb.prange` and scale with cores.

---

10. Key Design Principle: Separate I/O from Compute

PyArrow domain	Numba domain
Read Parquet files	Never touches Parquet
Column pruning	Operates on flat NumPy arrays
Predicate pushdown	No Python objects in hot loops

Zero-copy to NumPy	No strings, no dicts
Write output Parquet	Pure numeric computation
Schema management	Cache-compiled, reused across products and days

The bridge is always: `pa.Array.to_numpy(zero_copy_only=False)` for each column, done once per phase, then Numba takes over.

---

11. Approved? Next Steps

If this architecture is acceptable, I will implement in this order:

1. **numba_kernels.py** — all 7 kernels with type signatures and unit tests
2. **daily_pipeline.py** — single product, single day, end-to-end
3. **Synthetic data generator** — to validate without real CME data
4. **100-product orchestrator** — sequential loop with logging
5. **phase_f_aggregate.py** — multi-day pooling
6. **fan_chart.py** — matplotlib visualization